

模拟专题训练（简单题）全部是通过
模拟题中要求的过程，使用代码实现

模拟

2582. 连续头

给你一个长度为 n 的数组 a ，其中 a_i 表示第 i 个人的得分。如果存在一个长度为 k 的连续子数组，使得这个子数组中所有人的得分之和等于 k ，那么我们就称这个子数组为“连续头”。

请你求出数组 a 中“连续头”的个数。

输入格式

第一行包含两个整数 n 和 k 。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ 。

输出格式

输出一个整数，表示“连续头”的个数。

```
1 // 模拟实现
2 class Solution {
3 public:
4     int countSubarrays(vector<int>& a, int k) {
5         int n = a.size();
6         int ans = 0;
7         for (int i = 0; i < n; i++) {
8             int sum = 0;
9             for (int j = i; j < n; j++) {
10                 sum += a[j];
11                 if (sum == k) {
12                     ans++;
13                 }
14             }
15         }
16         return ans;
17     }
18 };
19 
```

844. 比较含退格的字符串

给定 s 和 t 两个字符串，当它们分别消除一些字符后可以获得相同的字符串时，我们称它们为“相等的字符串”。

消除字符的规则是：在字符串中出现退格符 $'#'$ 时，则删除该字符前一个字符；当字符串为空时，删除退格符本身；除此之外，其他字符都将保留。

例如， $s = "ab#c"$ ， $t = "ad#c"$ ，消除后都将变成 $"ac"$ 。

另举例子， $s = "ab##c"$ ， $t = "ab#c"$ ，消除后都将变成 $"c"$ 。

再举例子， $s = "a#c"$ ， $t = "b"$ ，消除后 s 会变成 $"c"$ ， t 会变成 $"b"$ ，它们不相等。

请你判断 s 和 t 是否相等。

输入格式

第一行包含两个字符串 s 和 t 。

输出格式

如果 s 和 t 相等，输出 $"true"$ ；否则，输出 $"false"$ 。

```
1 // 模拟实现
2 class Solution {
3 public:
4     bool backspaceCompare(string s, string t) {
5         int n = s.size(), m = t.size();
6         int i = n - 1, j = m - 1;
7         while (i >= 0 || j >= 0) {
8             while (i >= 0 && (s[i] == '#' || s[i] == ' ')) i--;
9             while (j >= 0 && (t[j] == '#' || t[j] == ' ')) j--;
10             if (i < 0 && j >= 0) return false;
11             if (j < 0 && i >= 0) return false;
12             if (i >= 0 && j >= 0 && s[i] != t[j]) return false;
13             i--; j--;
14         }
15         return true;
16     }
17 };
18 
```

867. 转置矩阵

给你一个二维矩阵 $matrix$ ，请你返回 $matrix$ 的转置矩阵。

转置矩阵的定义是：将原矩阵的行变为列，列变为行。

例如， $matrix = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ ，那么它的转置矩阵为 $\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$ 。

输入格式

第一行包含两个整数 m 和 n ，分别表示矩阵的行数和列数。

接下来 m 行，每行包含 n 个整数，表示矩阵 $matrix$ 。

输出格式

输出一个二维数组，表示 $matrix$ 的转置矩阵。

```
1 // 模拟实现
2 class Solution {
3 public:
4     vector<vector<int>> transpose(vector<vector<int>>& matrix) {
5         int m = matrix.size(), n = matrix[0].size();
6         vector<vector<int>> ans(n, vector<int>(m, 0));
7         for (int i = 0; i < m; i++) {
8             for (int j = 0; j < n; j++) {
9                 ans[j][i] = matrix[i][j];
10             }
11         }
12         return ans;
13     }
14 };
15 
```

2500. 删除每行中的最大值

给你一个 n 行 m 列的二维数组 $grid$ ，请你删除每一行中的最大值，并返回被删除元素之和。

例如， $grid = \begin{bmatrix} 1 & 2 & 4 \\ 3 & 3 & 1 \end{bmatrix}$ ，那么删除后的数组为 $\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$ ，被删除元素之和为 $4 + 1 = 5$ 。

输入格式

第一行包含两个整数 n 和 m 。

接下来 n 行，每行包含 m 个整数，表示数组 $grid$ 。

输出格式

输出一个整数，表示被删除元素之和。

```
1 // 模拟实现
2 class Solution {
3 public:
4     int deleteGreatestValue(vector<vector<int>>& grid) {
5         int n = grid.size(), m = grid[0].size();
6         for (int i = 0; i < m; i++) {
7             sort(grid[i].begin(), grid[i].end(), greater<int>());
8         }
9         int ans = 0;
10         for (int i = 0; i < n; i++) {
11             for (int j = 0; j < m; j++) {
12                 ans += grid[i][j];
13             }
14         }
15         return ans;
16     }
17 };
18 
```

3483. 不同三位数的数目

给你一个长度为 n 的数组 a ，其中 a_i 表示第 i 个数字。请你求出由 a 中的数字组成的不同三位数的数目。

例如， $a = [1, 2, 3]$ ，那么由 a 中的数字组成的三位数有 $123, 132, 213, 231, 312, 321$ ，共 6 个。

输入格式

第一行包含一个整数 n 。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ 。

输出格式

输出一个整数，表示不同三位数的数目。

```
1 // 模拟实现
2 class Solution {
3 public:
4     int countNumbers(vector<int>& a) {
5         int n = a.size();
6         int ans = 0;
7         for (int i = 0; i < n; i++) {
8             for (int j = 0; j < n; j++) {
9                 for (int k = 0; k < n; k++) {
10                     if (i != j && j != k && i != k) {
11                         ans++;
12                     }
13                 }
14             }
15         }
16         return ans;
17     }
18 };
19 
```

657. 机器人能否返回原点

在二维平面上，有一个机器人从原点 $(0, 0)$ 开始。给出它的移动顺序，判断这个机器人是否最终返回原点。

移动顺序由字符串 $moves$ 表示，其中 m 表示移动到 $(-1, 0)$ ， u 表示移动到 $(1, 0)$ ， d 表示移动到 $(0, -1)$ ， u 表示移动到 $(0, 1)$ 。

例如， $moves = "ud"$ ，机器人先移动到 $(-1, 0)$ ，再移动到 $(0, 1)$ ，最终返回原点。

输入格式

第一行包含一个字符串 $moves$ 。

输出格式

如果机器人最终返回原点，输出 $"true"$ ；否则，输出 $"false"$ 。

```
1 // 模拟实现
2 class Solution {
3 public:
4     bool returnToOrigin(string moves) {
5         int x = 0, y = 0;
6         for (char c : moves) {
7             if (c == 'u') x--;
8             if (c == 'd') x++;
9             if (c == 'l') y--;
10             if (c == 'r') y++;
11         }
12         return x == 0 && y == 0;
13     }
14 };
15 
```

1103. 分糖果 II

给你一个整数 $candies$ ，表示糖果的总数。还有一个整数 k ，表示糖果的份数。请你求出每份糖果的数量。

例如， $candies = 10$ ， $k = 3$ ，那么每份糖果的数量为 3 ，还剩下 1 个糖果。

输入格式

第一行包含两个整数 $candies$ 和 k 。

输出格式

输出一个整数，表示每份糖果的数量。

```
1 // 模拟实现
2 class Solution {
3 public:
4     int distributeCandies(int candies, int k) {
5         return candies / k;
6     }
7 };
8 
```

1688. 比赛中的配对次数

给你一个整数 n ，表示比赛的总人数。比赛采用淘汰制，每场比赛淘汰一人。请你求出比赛的配对次数。

例如， $n = 4$ ，那么比赛的配对次数为 3 。

输入格式

第一行包含一个整数 n 。

输出格式

输出一个整数，表示比赛的配对次数。

```
1 // 模拟实现
2 class Solution {
3 public:
4     int numberOfMatches(int n) {
5         return n - 1;
6     }
7 };
8 
```